

Лабораторная работа №4

Содержание

1. Цель работы 🎯	3
2. Теоретические сведения 📖	4
3. Порядок выполнения работы 📋	5
3.1. Первичная настройка базы данных	5
3.2. Запуск проекта	5
4. Исходный код и комментарии ⚙️	6
4.1. RelayCommand.cs — реализация команды MVVM	6
4.2. StoreDbContext.cs — контекст базы данных	7
4.3. CartItem.cs — элемент корзины	8
4.4. MainViewModel.cs — основная логика взаимодействия с UI	9
4.5. Репозиторий ProductRepository.cs — операции с товарами	11
4.6. Репозиторий OrderRepository.cs — работа с заказами	13
4.7. ProductViewModel.cs — управление данными товаров	14
4.8. OrderViewModel.cs — управление заказами	17
5. Работа с XAML-интерфейсом </>	20
5.1. Привязка контекста к ViewModel	20
5.2. Отображение списка продуктов	20
5.3. Команды кнопок	20
6. Примеры интерфейса пользователя 🗑️	21
7. Контрольные вопросы ❓	24

List of Listings

4.1	Класс RelayCommand	7
4.2	Контекст базы данных	8
4.3	Листинг «Корзины товаров»	8
4.4	Логика взаимодействия с UI	11
4.5	Основные операции с товарами	12
4.6	Репозиторий OrderRepository	14
4.7	Код класса ProductViewModel	16
4.8	Код класса OrderViewModel	19
5.1	Привязка контекста к ViewModel	20
5.2	Отображение списка продуктов	20
5.3	Привязка кнопок к соответствующим командам ViewModel	20

1. Цель работы

Освоить принципы построения приложений на WPF с применением паттерна MVVM, базы данных SQLite и технологии Entity Framework Core. Разработать функциональность управления товарами и заказами в рамках настольного приложения.

Задание:

1. Изучить архитектуру MVVM и реализовать её на практике.
2. Использовать Entity Framework Core для доступа к базе данных.
3. Организовать взаимодействие между слоями модели, представления и логики представления.
4. Настроить графический интерфейс пользователя для работы с товарами и заказами.
5. Реализовать команды добавления, редактирования и удаления данных.

Весь код доступен в [репозитории на GitHub](#) ¹

¹Ссылки, выделенные голубым цветом, ведут на внешние ресурсы; ссылки, выделенные светло-синим цветом, указывают на исходный код в репозитории GitHub .

2. Теоретические сведения

StoreManager — это настольное WPF-приложение на платформе .NET, реализующее базовую функциональность управления магазином: добавление/редактирование товаров, формирование заказов, управление корзиной, отображение и удаление заказов. Проект построен по принципам MVVM (Model-View-ViewModel), использует Entity Framework Core для работы с SQLite базой данных.

Основные компоненты:

- Model: классы `Product`, `Order`, `OrderItem`, `CartItem` — описывают бизнес-логику и данные.
- ViewModel: классы `MainViewModel`, `ProductViewModel`, `OrderViewModel` управляют состоянием и взаимодействием между представлением и моделью.
- View: XAML-интерфейс, представляющий UI, привязанный к ViewModel.

Используемые технологии:

- C#, .NET
- WPF (XAML)
- Entity Framework Core
- SQLite
- MVVM-паттерн

3. Порядок выполнения работы №

В данном разделе последовательно описан процесс подготовки, запуска и анализа исходного кода проекта. Необходимо **строго** придерживаться шагов, описанных ниже, чтобы успешно воспроизвести и исследовать функциональность приложения.

1. Для начала необходимо открыть проект в среде разработки Visual Studio.
2. Далее необходимо установить требуемые библиотеки через NuGet Package Manager. Список необходимых NuGet-пакетов:
 - Microsoft.EntityFrameworkCore
 - Microsoft.EntityFrameworkCore.Sqlite
 - Microsoft.Data.Sqlite

3.1. Первичная настройка базы данных

После установки зависимостей необходимо выполнить инициализацию базы данных. При первом запуске приложение автоматически создаёт файл базы данных `store.db` с помощью метода `EnsureCreated()`.

3.2. Запуск проекта

После выполнения всех предыдущих шагов необходимо запустить приложение. Проект `StoreManager_4lab` должен быть выбран в качестве стартового.

4. Исходный код и комментарии ⚙️

В данном разделе рассмотрены ключевые компоненты проекта, реализующие бизнес-логику и связь с пользовательским интерфейсом. Далее последовательно представлены листинги кода с пояснениями.

4.1. RelayCommand.cs — реализация команды MVVM

```

4 namespace StoreManager_4lab.ViewModels
5 {
6     public class RelayCommand : ICommand
7     {
8         private readonly Action<object> _execute;
9         //private readonly Predicate<object> _canExecute;
10
11         private readonly Func<object, bool> _canExecute;
12
13
14
15         public RelayCommand(Action<object> execute, Func<object, bool> canExecute = null)
16         {
17             _execute = execute ?? throw new ArgumentNullException(nameof(execute));
18             _canExecute = canExecute;
19         }
20
21         public event EventHandler CanExecuteChanged
22         {
23             add { CommandManager.RequerySuggested += value; }
24             remove { CommandManager.RequerySuggested -= value; }
25         }
26
27         public bool CanExecute(object parameter) => _canExecute?.Invoke(parameter) ?? true;
28
29         public void Execute(object parameter) => _execute(parameter);
30     }
31 }
32 public class RelayCommand : ICommand
33 {
34     private readonly Action<object> _execute; // делегат выполнения команды
35     private readonly Func<object, bool> _canExecute; // делегат проверки возможности выполнения
36     ↪ команды
37
38     // Конструктор команды
39     public RelayCommand(Action<object> execute, Func<object, bool> canExecute = null)
40     {
41         _execute = execute ?? throw new ArgumentNullException(nameof(execute)); // инициализация
42         ↪ метода выполнения
43         _canExecute = canExecute; // инициализация метода проверки доступности

```

```
42     }
43
44     // Событие, которое вызывается при изменении условий выполнения команды
45     public event EventHandler CanExecuteChanged
46     {
47         add { CommandManager.RequerySuggested += value; }
48         remove { CommandManager.RequerySuggested -= value; }
49     }
50
51     // Проверка, может ли команда быть выполнена
52     public bool CanExecute(object parameter) => _canExecute?.Invoke(parameter) ?? true;
53
54     // Выполнение команды
55     public void Execute(object parameter) => _execute(parameter);
56 }
```

Листинг 4.1 – Класс RelayCommand

В листинге 4.1 представлен класс RelayCommand. Данный класс реализует интерфейс ICommand, что позволяет привязывать действия в UI к методам ViewModel. Применяется для реализации кнопок и команд.

1. RelayCommand(...) — конструктор команды. Принимает делегаты выполнения и условия выполнения. Используется при создании команд в ViewModel.
2. CanExecute(object parameter) — возвращает true, если команду можно выполнить. Применяется для активации/деактивации элементов интерфейса.
3. Execute(object parameter) — запускает делегат действия команды. Вызывается при клике на кнопку или другое действие пользователя.

4.2. StoreDbContext.cs — контекст базы данных

```
1 public class StoreDbContext : DbContext
2 {
3     // Таблица товаров
4     public DbSet<Product> Products { get; set; }
5     // Таблица заказов
6     public DbSet<Order> Orders { get; set; }
7     // Таблица позиций заказа
8     public DbSet<OrderItem> OrderItems { get; set; }
9     // Конфигурация подключения к базе данных
10    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
11    {
12        optionsBuilder.UseSqlite("Data Source=store.db"); // указание строки подключения к SQLite
13    }
14    // Настройка модели данных
```

```
15     protected override void OnModelCreating(ModelBuilder modelBuilder)
16     {
17         modelBuilder.Entity<Order>()
18             .HasMany(o => o.Items) // один заказ содержит множество позиций
19             .WithOne()             // без явной обратной связи
20             .OnDelete(DeleteBehavior.Cascade); // при удалении заказа удаляются все его позиции
21     }
22 }
```

Листинг 4.2 – Контекст базы данных

1. `OnConfiguring(DbContextOptionsBuilder optionsBuilder)` — вызывается при создании экземпляра контекста. Устанавливает строку подключения к базе данных SQLite.
2. `OnModelCreating(ModelBuilder modelBuilder)` — выполняется при инициализации модели EF. Здесь настраивается каскадное удаление зависимых сущностей (например, позиций заказа при удалении заказа). Определяет три таблицы: `Product`, `Order`, `OrderItem`.

4.3. CartItem.cs — элемент корзины

Далее представлен листинг класса `CartItem`, отвечающего за временное хранение товаров (Листинг 4.3).

```
1 public class CartItem : INotifyPropertyChanged
2 {
3     public Product Product { get; set; } // ссылка на продукт
4
5     private int _quantity; // поле для хранения количества
6
7     public int Quantity
8     {
9         get => _quantity; // получение значения
10        set { _quantity = value; OnPropertyChanged(); } // установка значения и уведомление об
11        ↪ изменении
12    }
13
14    // Событие, уведомляющее об изменении свойства
15    public event PropertyChangedEventHandler PropertyChanged;
16
17    // Метод вызова события изменения свойства
18    protected void OnPropertyChanged([CallerMemberName] string name = null) =>
19        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
20 }
```

Листинг 4.3 – Листинг «Корзины товаров»

CartItem служит для временного хранения товаров в корзине перед созданием заказа. Используется в ObservableCollection<CartItem>.

1. Product — ссылка на экземпляр товара, связанный с объектом корзины.
2. Quantity — количество выбранного товара в корзине. При изменении уведомляет UI через механизм INotifyPropertyChanged.
3. OnPropertyChanged(string name) — метод, вызывающий событие PropertyChanged для обновления интерфейса при изменении свойств.

4.4. MainViewModel.cs — основная логика взаимодействия с UI

```
1 public class MainViewModel : INotifyPropertyChanged
2 {
3     // Контекст базы данных
4     private readonly StoreDbContext _context = new();
5
6     // Коллекция всех товаров
7     public ObservableCollection<Product> Products { get; set; } = new();
8
9     // Коллекция всех заказов
10    public ObservableCollection<Order> Orders { get; set; } = new();
11
12    // Коллекция товаров, добавленных в корзину
13    public ObservableCollection<CartItem> CartItems { get; } = new();
14
15    // Команда добавления товара
16    public ICommand AddProductCommand { get; }
17
18    // Команда создания заказа
19    public ICommand AddOrderCommand { get; }
20
21    // Конструктор ViewModel
22    public MainViewModel()
23    {
24        // Инициализация команд
25        AddProductCommand = new RelayCommand(_ => AddProduct(), _ => CanAddProduct());
26        AddOrderCommand = new RelayCommand(_ => AddOrder());
27
28        // Создание базы данных при первом запуске
29        _context.Database.EnsureCreated();
30
31        // Загрузка данных из базы
32        LoadProducts();
33        LoadOrders();
34    }
35
```

```
36 // Загрузка всех товаров из базы данных
37 private void LoadProducts()
38 {
39     Products.Clear();
40     foreach (var p in _context.Products.ToList())
41         Products.Add(p);
42 }
43
44 // Загрузка всех заказов с товарами из базы данных
45 private void LoadOrders()
46 {
47     Orders.Clear();
48     var allOrders = _context.Orders
49         .Include(o => o.Items)
50         .ThenInclude(i => i.Product)
51         .ToList();
52
53     foreach (var order in allOrders)
54         Orders.Add(order);
55 }
56
57 // Добавление нового товара
58 private void AddProduct()
59 {
60     var product = new Product
61     {
62         Name = "Новый товар", // название товара
63         Price = 100, // цена по умолчанию
64         Stock = 10, // количество по умолчанию
65         Category = "Общая" // категория товара
66     };
67
68     _context.Products.Add(product); // добавление в базу данных
69     _context.SaveChanges(); // сохранение изменений
70     LoadProducts(); // обновление списка товаров
71 }
72
73 // Добавление нового заказа
74 private void AddOrder()
75 {
76     if (!CartItems.Any()) return; // проверка наличия товаров в корзине
77
78     var order = new Order
79     {
80         CustomerName = "Клиент", // имя покупателя
81         OrderDate = DateTime.Now, // дата оформления
82         Items = CartItems.Select(c => new OrderItem
83         {
84             Product = c.Product, // товар
85             Quantity = c.Quantity // количество
86         }).ToList()
```

```
87         };
88
89         _context.Orders.Add(order); // добавление заказа в БД
90         _context.SaveChanges();     // сохранение
91         LoadOrders();               // обновление списка
92         CartItems.Clear();           // очистка корзины
93     }
94
95     // Условие доступности команды добавления товара
96     private bool CanAddProduct() => true;
97
98     public event PropertyChangedEventHandler PropertyChanged;
99
100    // Обработка события изменения свойства
101    protected void OnPropertyChanged([CallerMemberName] string name = null) =>
102        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
103 }
```

Листинг 4.4 – Логика взаимодействия с UI

В листинге 4.4 представлен код класса `MainViewModel`. Он связывает данные с пользовательским интерфейсом. В нём реализованы команды для добавления товаров и заказов, загрузка данных из базы и взаимодействие с коллекциями. Этот класс является ядром логики MVVM и предоставляет интерфейс между XAML и данными модели.

1. `MainViewModel()` — конструктор класса. Производит инициализацию базы, загрузку данных и настройку команд.
2. `LoadProducts()` — загружает все товары из базы данных в `ObservableCollection` для отображения.
3. `LoadOrders()` — загружает все заказы и их связанные товары.
4. `AddProduct()` — добавляет новый товар в базу и обновляет отображаемый список.
5. `AddOrder()` — формирует и сохраняет заказ на основе содержимого корзины.
6. `CanAddProduct()` — всегда возвращает `true`, используется как условие для активности команды добавления товара.
7. `OnPropertyChanged(string name)` — реализует уведомление об изменении свойства, используется в привязках.

4.5. Репозиторий ProductRepository.cs — операции с товарами

```
1 public class ProductRepository
```

```
2 {
3     private const string ConnectionString = "Data Source=store.db"; // строка подключения к базе
        ↳ данных
4
5     public List<Product> GetAll()
6     {
7         var list = new List<Product>(); // список для хранения всех продуктов
8         using var connection = new SqliteConnection(ConnectionString); // открытие соединения с
        ↳ базой данных
9         connection.Open(); // выполнение подключения к базе данных
10
11        var command = connection.CreateCommand(); // создание SQL-команды
12        command.CommandText = "SELECT * FROM Product"; // выборка всех записей из таблицы товаров
13
14        using var reader = command.ExecuteReader(); // выполнение команды и получение результатов
15        while (reader.Read()) // построчное чтение результатов
16        {
17            list.Add(new Product // добавление экземпляра товара в список
18            {
19                Id = reader.GetInt32(0), // идентификатор товара
20                Name = reader.GetString(1), // название товара
21                Price = reader.GetDecimal(2), // цена товара
22                Stock = reader.GetInt32(3), // остаток товара на складе
23                Category = reader.IsDBNull(4) ? null : reader.GetString(4) // категория товара
        ↳ (может отсутствовать)
24            });
25        }
26        return list; // возвращение списка всех товаров
27    }
28
29    public void Add(Product product)
30    {
31        using var connection = new SqliteConnection(ConnectionString); // открытие соединения с
        ↳ базой данных
32        connection.Open(); // выполнение подключения к БД
33
34        var command = connection.CreateCommand(); // создание SQL-команды
35        command.CommandText = "INSERT INTO Product (Name, Price, Stock, Category) VALUES (@name,
        ↳ @price, @stock, @category)"; // вставка нового товара в таблицу Product
36        command.Parameters.AddWithValue("@name", product.Name); // название товара
37        command.Parameters.AddWithValue("@price", product.Price); // цена товара
38        command.Parameters.AddWithValue("@stock", product.Stock); // количество товара на складе
39        command.Parameters.AddWithValue("@category", product.Category ?? (object)DBNull.Value); //
        ↳ категория товара (или null)
40
41        command.ExecuteNonQuery(); // выполнение SQL-команды
42    }
43 }
```

Класс `ProductRepository` предоставляет низкоуровневый доступ к таблице товаров. В нём реализованы методы чтения и добавления продуктов с использованием SQLite.

1. `GetAll()` — выполняет подключение к базе данных и извлекает все товары из таблицы `Product`. Используется при загрузке списка товаров в интерфейс.
2. `Add(Product product)` — добавляет новый товар в базу данных с помощью SQL-запроса `INSERT`. Используется при создании новых товаров пользователем.

4.6. Репозиторий `OrderRepository.cs` — работа с заказами

```
1 public class OrderRepository
2 {
3     private const string ConnectionString = "Data Source=store.db"; // строка подключения к БД
4
5     public List<Order> GetAll(List<Product> allProducts)
6     {
7         var orders = new List<Order>(); // список заказов
8         using var connection = new SqliteConnection(ConnectionString); // открытие соединения с
9         // базой данных
10        connection.Open(); // выполнение подключения
11
12        var command = connection.CreateCommand(); // создание SQL-команды
13        command.CommandText = "SELECT Id, CustomerName, OrderDate FROM [Order]"; // выборка данных
14        // заказов
15        using var reader = command.ExecuteReader(); // выполнение запроса и чтение результатов
16
17        while (reader.Read()) // построчное чтение результата
18        {
19            var order = new Order
20            {
21                Id = reader.GetInt32(0), // идентификатор заказа
22                CustomerName = reader.GetString(1), // имя клиента
23                OrderDate = DateTime.Parse(reader.GetString(2)), // дата оформления заказа
24                Items = new List<OrderItem>() // список товаров в заказе
25            };
26            orders.Add(order); // добавление заказа в коллекцию
27        }
28
29        foreach (var order in orders) // обход заказов для загрузки позиций
30        {
31            var itemCmd = connection.CreateCommand(); // создание SQL-команды для выбора позиций
32            // заказа
33            itemCmd.CommandText = "SELECT ProductId, Quantity FROM OrderItem WHERE OrderId =
34            // @orderId"; // выборка товарных позиций
35            itemCmd.Parameters.AddWithValue("@orderId", order.Id); // параметр: идентификатор заказа
36            using var itemReader = itemCmd.ExecuteReader(); // выполнение запроса
37        }
38    }
39 }
```

```
34         while (itemReader.Read()) // построчное чтение позиций
35         {
36             var productId = itemReader.GetInt32(0); // идентификатор товара
37             var quantity = itemReader.GetInt32(1); // количество товара
38             var product = allProducts.FirstOrDefault(p => p.Id == productId); // поиск товара в
               ↳ общем списке
39
40             if (product != null)
41             {
42                 order.Items.Add(new OrderItem // добавление товара в список позиций заказа
43                 {
44                     Product = product,
45                     Quantity = quantity
46                 });
47             }
48         }
49     }
50     return orders; // возвращение списка заказов
51 }
52 }
```

Листинг 4.6 – Репозиторий OrderRepository

В листинге 4.6 представлен код класса `OrderRepository`, который позволяет загружать заказы с их товарами из базы данных. Используется прямое взаимодействие с `SQLite` для выборки и объединения данных. Этот репозиторий играет роль интерфейса между бизнес-логикой и хранилищем заказов.

1. `GetAll(List<Product> allProducts)` — извлекает список всех заказов из таблицы `Order`, а также загружает связанные позиции из таблицы `OrderItem`.
2. `Add(Order order)` — добавляет новый заказ и его позиции в базу данных в рамках транзакции. Используется при оформлении нового заказа.
3. `Delete(int orderId)` — удаляет заказ по идентификатору из таблицы `Order`. При этом связанные позиции удаляются автоматически (если настроено каскадное удаление).

4.7. ProductViewModel.cs — управление данными товаров

```
1 public class ProductViewModel : BaseViewModel
2 {
3     private Product _selectedProduct; // выбранный товар
4     private readonly StoreDbContext _context; // контекст базы данных
5     private string _filterCategory; // выбранная категория для фильтрации
6
7     public ObservableCollection<Product> Products { get; } = new(); // список всех товаров
8     public ObservableCollection<string> Categories { get; } = new(); // список доступных категорий
9 }
```

```
10 // Свойство выбранного товара
11 public Product SelectedProduct
12 {
13     get => _selectedProduct;
14     set => SetProperty(ref _selectedProduct, value);
15 }
16
17 // Свойство фильтрации по категории
18 public string FilterCategory
19 {
20     get => _filterCategory;
21     set
22     {
23         if (SetProperty(ref _filterCategory, value))
24         {
25             LoadProducts(); // при изменении категории перезагружается список
26         }
27     }
28 }
29
30 // Команды управления
31 public ICommand AddProductCommand { get; }
32 public ICommand UpdateProductCommand { get; }
33 public ICommand DeleteProductCommand { get; }
34
35 // Конструктор
36 public ProductViewModel()
37 {
38     _context = new StoreDbContext(); // инициализация контекста
39     _context.Database.EnsureCreated(); // создание базы при необходимости
40
41     LoadCategories(); // загрузка всех категорий
42     LoadProducts(); // загрузка всех товаров
43
44     // инициализация команд
45     AddProductCommand = new RelayCommand(_ => AddProduct());
46     UpdateProductCommand = new RelayCommand(_ => UpdateProduct(), _ => SelectedProduct != null);
47     DeleteProductCommand = new RelayCommand(_ => DeleteProduct(), _ => SelectedProduct != null);
48 }
49
50 // Загрузка всех товаров (с фильтрацией, если указана категория)
51 private void LoadProducts()
52 {
53     Products.Clear();
54     var products = _context.Products.AsQueryable();
55
56     if (!string.IsNullOrEmpty(FilterCategory))
57         products = products.Where(p => p.Category == FilterCategory);
58
59     foreach (var product in products.ToList())
60         Products.Add(product);
```

```
61     }
62
63     // Загрузка уникальных категорий товаров
64     private void LoadCategories()
65     {
66         Categories.Clear();
67         var categories = _context.Products.Select(p => p.Category).Distinct().ToList();
68         foreach (var category in categories)
69             Categories.Add(category);
70     }
71
72     // Добавление нового товара
73     private void AddProduct()
74     {
75         var product = new Product { Name = "Новый товар", Price = 0, Stock = 0, Category = "Общая"
76             ↪ };
77         _context.Products.Add(product);
78         _context.SaveChanges();
79         Products.Add(product);
80         SelectedProduct = product;
81         LoadCategories();
82     }
83
84     // Обновление выбранного товара
85     private void UpdateProduct()
86     {
87         _context.Products.Update(SelectedProduct);
88         _context.SaveChanges();
89         LoadProducts();
90     }
91
92     // Удаление выбранного товара
93     private void DeleteProduct()
94     {
95         if (SelectedProduct != null)
96         {
97             _context.Products.Remove(SelectedProduct);
98             _context.SaveChanges();
99             Products.Remove(SelectedProduct);
100             SelectedProduct = null;
101             LoadCategories();
102         }
103     }
```

Листинг 4.7 – Код класса ProductViewModel

Листинг 4.7 отражает код класса ProductViewModel. Эта модель управляет отображением и изменением информации о товарах. Поддерживает добавление, редактирование, удаление и фильтрацию по категориям. Используется для управления интерфейсом раздела «Товары».

1. `ProductViewModel()` — конструктор, инициализирует базу, загружает категории и товары, а также назначает команды.
2. `LoadProducts()` — загружает все товары из базы, с учётом установленной категории фильтрации.
3. `LoadCategories()` — извлекает уникальные категории товаров для отображения в выпадающем списке.
4. `AddProduct()` — создаёт новый товар, сохраняет его в базу и обновляет интерфейс.
5. `UpdateProduct()` — сохраняет изменения выбранного товара, и обновляет список.
6. `DeleteProduct()` — удаляет выбранный товар из базы и обновляет список, если товар не используется в заказах.

4.8. OrderViewModel.cs — управление заказами

```
1 public class OrderViewModel : BaseViewModel
2 {
3     private readonly StoreDbContext _context; // контекст базы данных
4     private Order _selectedOrder; // выбранный заказ
5     private string _customerName; // имя клиента
6
7     public ObservableCollection<Order> Orders { get; } = new(); // список заказов
8     public ObservableCollection<Product> AvailableProducts { get; } = new(); // доступные товары
9     public ObservableCollection<OrderItem> SelectedItems { get; } = new(); // позиции текущего
10    ↪ заказа
11    public string CustomerName
12    {
13        get => _customerName;
14        set => SetProperty(ref _customerName, value); // установка имени клиента
15    }
16    public Order SelectedOrder
17    {
18        get => _selectedOrder;
19        set
20        {
21            if (SetProperty(ref _selectedOrder, value) && value != null)
22            {
23                CustomerName = value.CustomerName; // установка имени из выбранного заказа
24                SelectedItems.Clear();
25                foreach (var item in value.Items)
26                    SelectedItems.Add(item); // заполнение списка позиций заказа
27            }
28        }
29    }
30    // Команды управления заказами
```

```
30     public ICommand AddOrderCommand { get; }
31     public ICommand UpdateOrderCommand { get; }
32     public ICommand DeleteOrderCommand { get; }
33     // Конструктор
34     public OrderViewModel()
35     {
36         _context = new StoreDbContext();
37         _context.Database.EnsureCreated();
38
39         LoadOrders(); // загрузка заказов из БД
40         LoadAvailableProducts(); // загрузка доступных товаров
41
42         AddOrderCommand = new RelayCommand(_ => AddOrder());
43         UpdateOrderCommand = new RelayCommand(_ => UpdateOrder(), _ => SelectedOrder != null);
44         DeleteOrderCommand = new RelayCommand(_ => DeleteOrder(), _ => SelectedOrder != null);
45     }
46     // Загрузка всех заказов с товарами
47     private void LoadOrders()
48     {
49         Orders.Clear();
50         foreach (var order in _context.Orders.Include(o => o.Items).ThenInclude(i =>
51             ↪ i.Product).ToList())
52             Orders.Add(order);
53     }
54     // Загрузка доступных товаров
55     private void LoadAvailableProducts()
56     {
57         AvailableProducts.Clear();
58         foreach (var product in _context.Products.ToList())
59             AvailableProducts.Add(product);
60     }
61     // Добавление нового заказа
62     private void AddOrder()
63     {
64         if (string.IsNullOrWhiteSpace(CustomerName) || SelectedItems.Count == 0) return;
65         var newOrder = new Order
66         {
67             CustomerName = CustomerName,
68             OrderDate = DateTime.Now,
69             Items = SelectedItems.Select(i => new OrderItem
70             {
71                 Product = i.Product,
72                 Quantity = i.Quantity
73             }).ToList()
74         };
75         _context.Orders.Add(newOrder);
76         _context.SaveChanges();
77         Orders.Add(newOrder); // добавление заказа в список
78     }
79     // Обновление существующего заказа
80     private void UpdateOrder()
```

```
80     {
81         if (SelectedOrder == null) return;
82         SelectedOrder.CustomerName = CustomerName;
83         SelectedOrder.Items = SelectedItems.Select(i => new OrderItem
84         {
85             Product = i.Product,
86             Quantity = i.Quantity
87         }).ToList();
88
89         _context.Orders.Update(SelectedOrder);
90         _context.SaveChanges();
91     }
92     // Удаление заказа
93     private void DeleteOrder()
94     {
95         if (SelectedOrder != null)
96         {
97             _context.Orders.Remove(SelectedOrder);
98             _context.SaveChanges();
99             Orders.Remove(SelectedOrder);
100             SelectedOrder = null;
101             SelectedItems.Clear();
102             CustomerName = string.Empty;
103         }
104     }
105 }
```

Листинг 4.8 – Код класса OrderViewModel

1. `OrderViewModel()` — конструктор. Создаёт контекст базы данных, загружает заказы и товары, и инициализирует команды.
2. `LoadOrders()` — загружает все существующие заказы с их товарными позициями.
3. `LoadAvailableProducts()` — загружает список товаров, доступных для добавления в заказ.
4. `AddOrder()` — создаёт новый заказ на основе текущего списка позиций и сохраняет его в базу.
5. `UpdateOrder()` — обновляет информацию выбранного заказа.
6. `DeleteOrder()` — удаляет выбранный заказ и сбрасывает связанные данные.
7. `OnPropertyChanged(...)` — уведомляет об изменениях свойств для обновления интерфейса.

5. Работа с XAML-интерфейсом </>

На завершающем этапе необходимо рассмотреть элементы графического интерфейса. Ниже представлены ключевые фрагменты XAML, демонстрирующие организацию интерфейса. Включены только ключевые элементы.

5.1. Привязка контекста к ViewModel

```
1 <Window.DataContext>
2   <vm:MainViewModel />
3 </Window.DataContext>
```

Листинг 5.1 – Привязка контекста к ViewModel

Листинг 5.1 отражает привязку контекста `Window.DataContext` к `ViewModel`. Это позволяет назначить `MainViewModel` как источник данных для всех привязок в окне.

5.2. Отображение списка продуктов

```
1 <ListBox ItemsSource="{Binding Products}" SelectedItem="{Binding SelectedProduct}">
2   <ListBox.ItemTemplate>
3     <DataTemplate>
4       <StackPanel Orientation="Horizontal">
5         <TextBlock Text="{Binding Name}" />
6         <TextBlock Text="{Binding Price}" />
7       </StackPanel>
8     </DataTemplate>
9   </ListBox.ItemTemplate>
10 </ListBox>
```

Листинг 5.2 – Отображение списка продуктов

На листинге 5.2 приведен пример отображения списка товаров с привязкой к `Products` и выбранным элементом — `SelectedProduct`.

5.3. Команды кнопок

Каждая кнопка привязана к определенному обработчику событий (методу), что позволяет выполнять необходимые операции при нажатии кнопки пользователем. Пример разметки для кнопок приведен на листинге 5.3.

```
1 <Button Content="Добавить" Command="{Binding AddProductCommand}" />
2 <Button Content="Обновить" Command="{Binding UpdateProductCommand}" />
3 <Button Content="Удалить" Command="{Binding DeleteProductCommand}" />
```

Листинг 5.3 – Привязка кнопок к соответствующим командам ViewModel

6. Примеры интерфейса пользователя 62

По завершению работы необходимо ознакомиться с внешним видом приложения. Ниже приведены фрагменты интерфейса с пояснениями. Ниже представлены иллюстрации ключевых элементов интерфейса (Рисунок 6.1 – 6.5):



Рисунок 6.1 – Главная форма приложения с разделами «Товары», «Корзина» и «Заказы».

Пользователь может добавлять новые товары, а также редактировать уже созданные с помощью соответствующих полей и кнопок. Для редактирования товара необходимо выбрать его в списке, нажать на него и его данные будут подставлены в соответствующие поля. После изменения информации необходимо нажать кнопку «Обновить». Для добавления нового товара необходимо ввести данные в соответствующие поля и нажать кнопку «Добавить».

При нажатии кнопки «В корзину» выбранные товары добавляются в блок «Корзина». У пользователя есть возможность управлять количеством товаров, увеличивая или уменьшая его. Ограничение – количество товаров на складе. Добавление товаров в корзину изображено на рисунке 6.2.

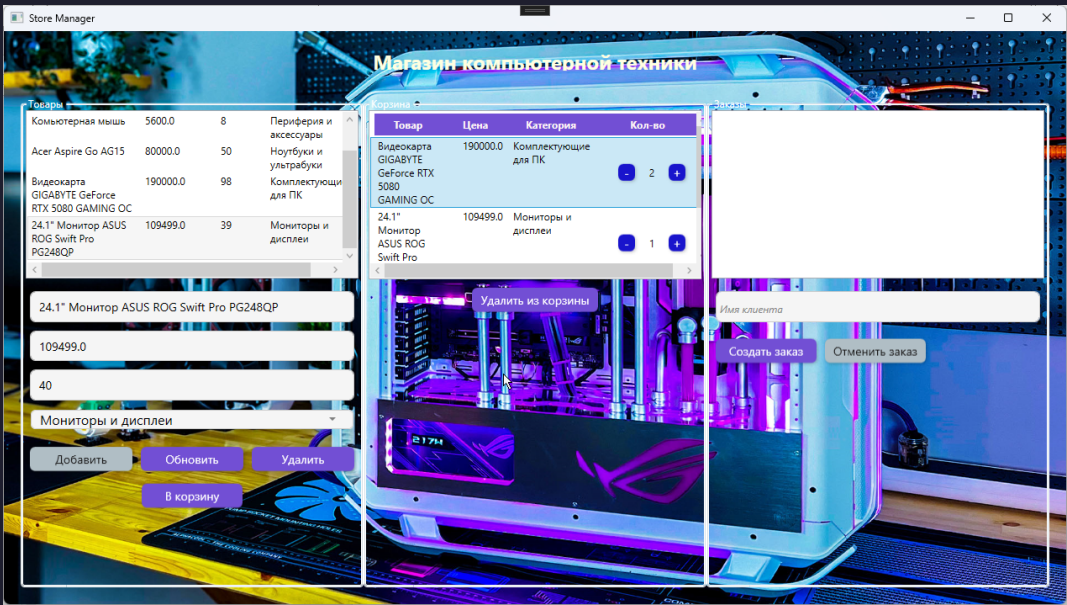


Рисунок 6.2 – Добавление товаров в «Корзину».

Также у пользователя есть возможность удалить товар из корзины с помощью кнопки «Удалить из корзины» (Рисунок 6.3). После удаления товара соответствующее количество товара вернется на склад.

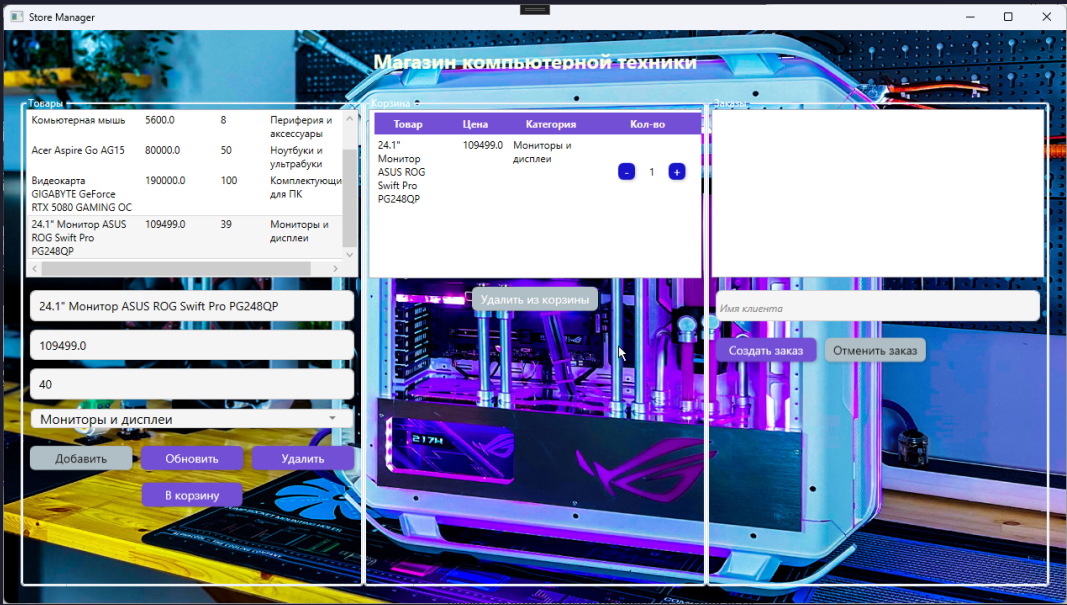


Рисунок 6.3 – Удаление выбранного товара.

После выбора необходимых товаров можно добавить новый заказ. Для создания заказа необходимо ввести имя клиента в соответствующее поле и нажать кнопку «Создать заказ» (Рисунок 6.4).

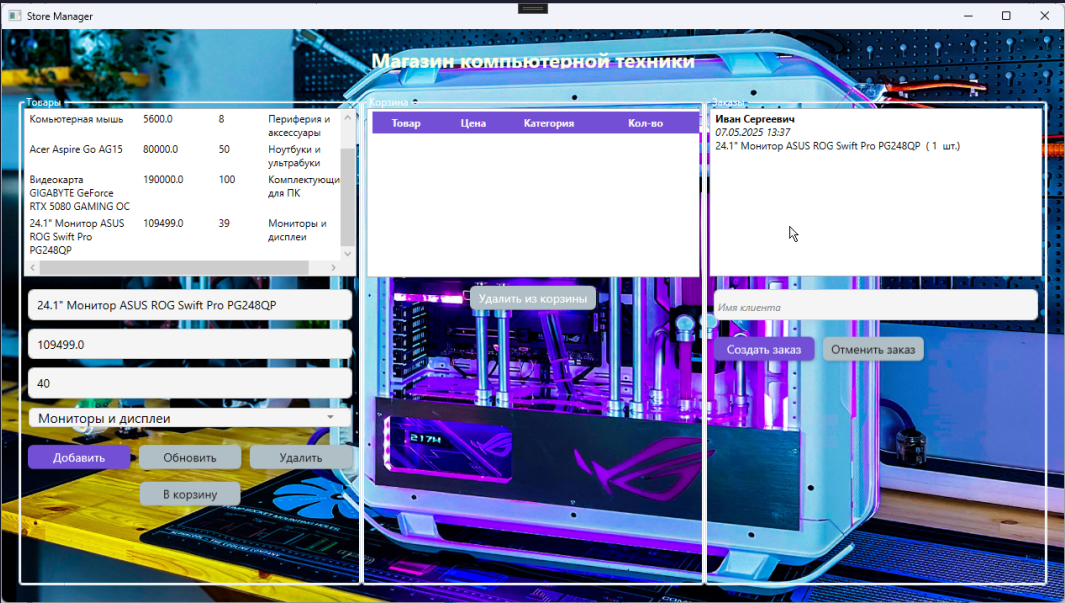


Рисунок 6.4 – Пример создания заказа.

При попытке добавить новый заказ не добавляя товары в корзину будет выведено соответствующее сообщение (Рисунок 6.5).

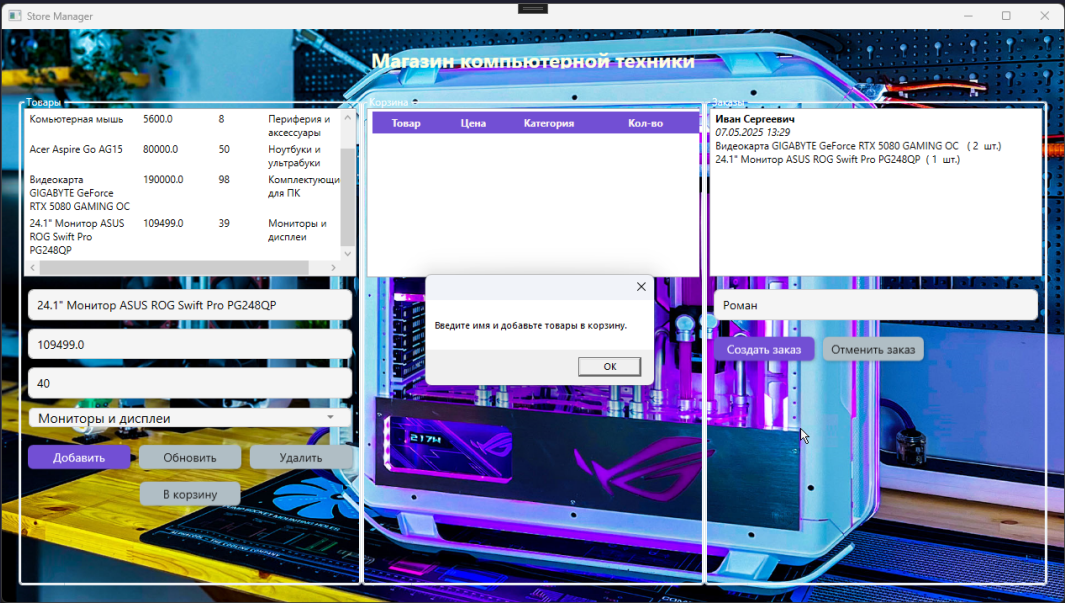


Рисунок 6.5 – Ошибка при создании заказа без товаров.

Пользователь может отменить заказ нажав на кнопку «Отменить заказ», после этого заказ удаляется из списка заказов.

7. Контрольные вопросы ?

1. **Что такое MVVM-паттерн и какова его структура в рамках WPF-приложения StoreManager?**

MVVM (Model-View-ViewModel) — это архитектурный шаблон, применяемый в WPF для разделения представления (XAML), логики отображения (ViewModel) и бизнес-логики (Model). В проекте StoreManager он представлен View (XAML), ViewModel-классами (MainViewModel, ProductViewModel, OrderViewModel) и моделями (Product, Order, OrderItem, CartItem).

2. **Как реализована связь между View и ViewModel в WPF-приложении? Приведите пример привязки.**

Связь реализована через механизм data binding. Пример: `Text= 'Binding ProductName'` привязывает значение поля к свойству во ViewModel. Контекст задаётся через `Window.DataContext`.

3. **Какие классы представляют модель предметной области (Model) в данном проекте и за что они отвечают?**

`Product` — товар, `Order` — заказ, `OrderItem` — позиция в заказе, `CartItem` — товар в корзине. Эти классы хранят бизнес-данные и реализуют интерфейс `INotifyPropertyChanged`.

4. **Какие задачи решает класс RelayCommand и как обеспечивается управление доступностью команд?**

Класс `RelayCommand` реализует интерфейс `ICommand` и позволяет связать действия UI с методами ViewModel. Доступность определяется функцией `CanExecute`, обновление — через событие `CanExecuteChanged`.

5. **Как работает механизм уведомления INotifyPropertyChanged в контексте классов Product, CartItem и ViewModel?**

При изменении свойства вызывается `OnPropertyChanged`, которое инициирует событие `PropertyChanged`. Это позволяет автоматически обновлять связанные элементы UI.

6. **Как осуществляется доступ к базе данных с помощью Entity Framework Core в StoreDbContext?**

Класс `StoreDbContext` наследуется от `DbContext` и предоставляет доступ к таблицам через `DbSet<>`. Настройки соединения и поведения моделей задаются в `OnConfiguring` и `OnModelCreating`.

7. **В чём различие между ProductRepository и OrderRepository по архитектуре и взаимодействию с БД?**

`ProductRepository` работает с одной таблицей `Product` и выполняет простые SQL-запросы. `OrderRepository` работает с двумя таблицами (`Order`, `OrderItem`) и управляет транзакциями и связями между сущностями.

8. Опишите этапы формирования нового заказа. Какие сущности при этом задействуются?

Пользователь добавляет товары в корзину (`CartItem`), затем оформляет заказ (`Order`), в который входят позиции (`OrderItem`) и клиентские данные. Все объекты сохраняются в базу через `StoreDbContext` или `OrderRepository`.

9. Какие команды реализованы в `MainViewModel`, и как они используются в интерфейсе пользователя?

Команды: `AddProductCommand`, `UpdateProductCommand`, `DeleteProductCommand`, `AddOrderCommand`, `DeleteOrderCommand`, `AddToCartCommand`, `RemoveFromCartCommand` и др. Каждая команда связана с кнопкой в UI и вызывает соответствующий метод `ViewModel`.

10. Как реализуется удаление заказа и восстановление остатка товара на складе при этом действии?

При удалении заказа вызывается метод `DeleteOrder`. Он возвращает количество товаров из заказа обратно в остатки соответствующих `Product`, после чего заказ удаляется из базы данных.